# real-tr-p8-semanticgloss

An AgentMPI run analysis

**Abstract**

This is the semantic-glossary ablation of the parallel book translation experiment: eight real Cursor subagents translate one book, and the `allreduce` that makes their terminology agree is performed by a model-implemented operator, `GLOSSARY_MERGE`, rather than by the exact, associative, idempotent `UNION` of `real-tr-p8-full`. The price of that substitution is exactly $p-1=7$ extra agent calls, one per internal node of the binomial tree, carrying 5,441 prompt and 4,873 completion tokens and costing $0.0894 — 95% of the $0.0939 total cost difference. The benefit is smaller than the harness's own documentation predicts and different in kind. Replaying `UNION` over this run's own eight term sheets, recovered from the blob store, yields a glossary with the *identical* 203 keys and only 7 differing values (3.4%): the semantic fold lost nothing, invented nothing, and never proposed a rendering no rank had proposed. Two of those seven are the one thing an exact operator cannot express — having chosen one transliteration of *Adler*, the operator propagated it to *Miss Adler* and *Miss Irene Adler*, keys set union treats as unrelated. The predicted population divergence did not occur and could not have: under `reduce_bcast` the root's answer is broadcast by handle, and all seven broadcast messages carry one digest, so every rank holds a byte-identical glossary. Neither did the 6.9× wall-time inflation come from the operator. Of this run's 1,152.9 s, 742 s is wall clock with no rank anywhere executing anything, waiting on worker attachment, against 2.3 s of such time in the exact run; and its 63.6% coordination share, the highest of the translation ablations, decomposes so that of the 5,861.7 rank-seconds blocked in collectives only 3.8% had a rank executing a merge while 72.4% elapsed at moments when nothing ran anywhere. At $p=8$ on this workload, then, semantic reduction is a linear-cost, logarithmic-latency way to buy a cross-key consistency that `UNION` cannot express, and its two advertised dangers — loss compounding with depth, and a population believing $p$ different glossaries — were both absent for reasons that are legible in the trace and that will not survive a change of algorithm or of scale.

## 1 What this run is

| property | value |
|---|---|
| run | `real-tr-p8-semanticgloss` |
| experiment | `translation` |
| campaign label | `real-tr` |
| declared world size | 8 |
| ranks appearing in the log | 8 |
| executors | `broker` × 8, `worker` × 56 |
| events | 398 |
| wall time | 1,152.90 s |
| job outcome | completed |

## 2 Headline measurements

| metric | value | meaning |
|---|---|---|
| achieved parallelism | 1.21× | total busy time over wall time |
| parallel efficiency | 15.2% | achieved parallelism per rank |
| idle fraction | 84.8% | rank-seconds paid for and unused |
| peak ranks busy | 8 | of 8 in the world |
| serial fraction of active time | 46.3% | time with exactly one rank busy |
| load imbalance | 1.35× | slowest rank's busy time over the mean |
| coordination share | 63.6% | rank-seconds blocked in collectives, of those available |
| coordination in flight | 88.1% | wall clock with any rank inside a collective |
| rank reattachments | 56 | fresh agent processes that took over a rank role |
| agent calls | 31 | invocations that reached a model |
| agent latency p50 / p95 | 59.52 / 388.87 s | per invocation |
| messages | 56 | 48 eager, 8 rendezvous |
| tokens sent | 31,906 | 8,893 deferred under rendezvous |
| tokens in / out | 48,530 / 13,878 | prompt and completion |
| cost | $0.3538 | 0.0255 $/kTok out |

## 3 What the analysis notices

- [critical] At least one rank waited 499 s for an agent to claim its task before the broker gave up. That is a pool-sizing failure outside the protocol, not a slow run.
- [warning] `halo_exchange/?` reports 0 messages but the fabric logged 14: the collective's own accounting disagrees with the traffic it produced.
- [note] Parallel efficiency is 15.2%: 84.8% of the rank-seconds paid for went unused.
- [warning] 1 collective(s) completed but recorded no duration (`halo_exchange`), so they contribute nothing to the coordination figures even though every participant blocked in them. The reported coordination share is short by exactly their blocking; it can be recovered from the event timestamps.
- [note] Collectives account for 63.6% of the run's rank-seconds, so more of the pool's time went to coordination than to work.
- [warning] The cost model's $\alpha$ and $\beta$ here are a median fallback, not a fit: the slope was negative ($\beta = -0.1084$ s/token), meaning latency fell as output grew — so something outside the model dominated the measurement, typically queueing, whose wait enters the latency but not the token count, on a fit with $R^2 = 0.032$. Any latency prediction from this run's calibration is a median, not a model.
- [ok] 56 rank reattachment(s) occurred, up to incarnation 9: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 9 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

- [warning] 1 of 9 collective(s) disagree with the cost model on *round* count while agreeing on messages: `barrier/central` recorded 0 against 2. Rounds are the critical-path term, so a disagreement here changes predicted latency without changing predicted traffic.

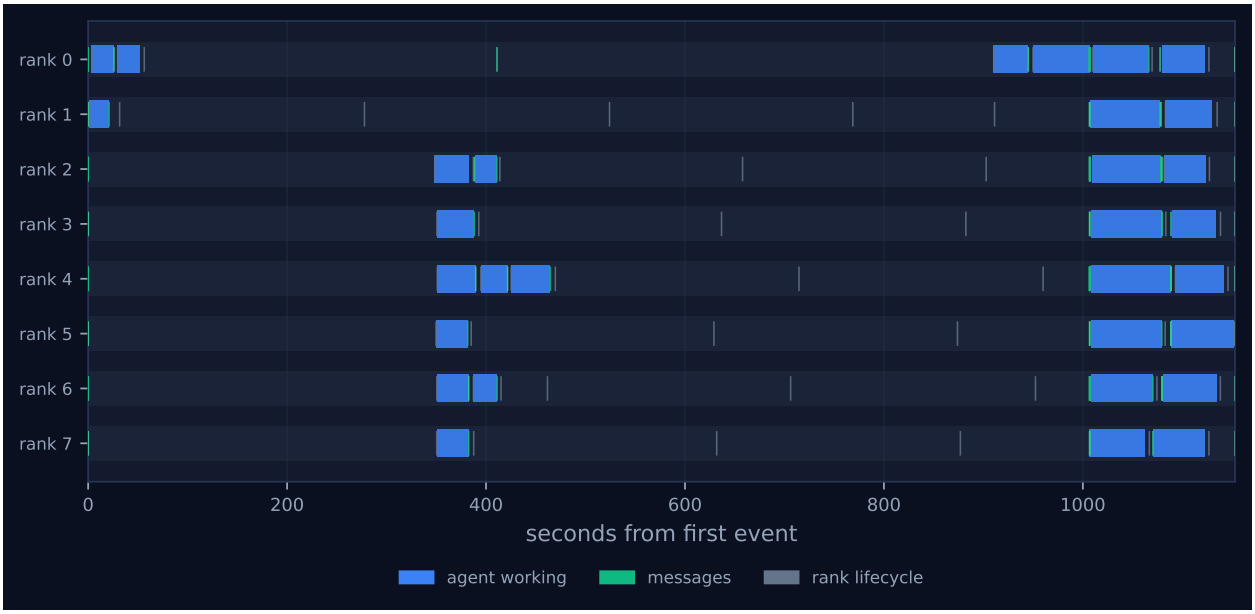


**Figure 1:** Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

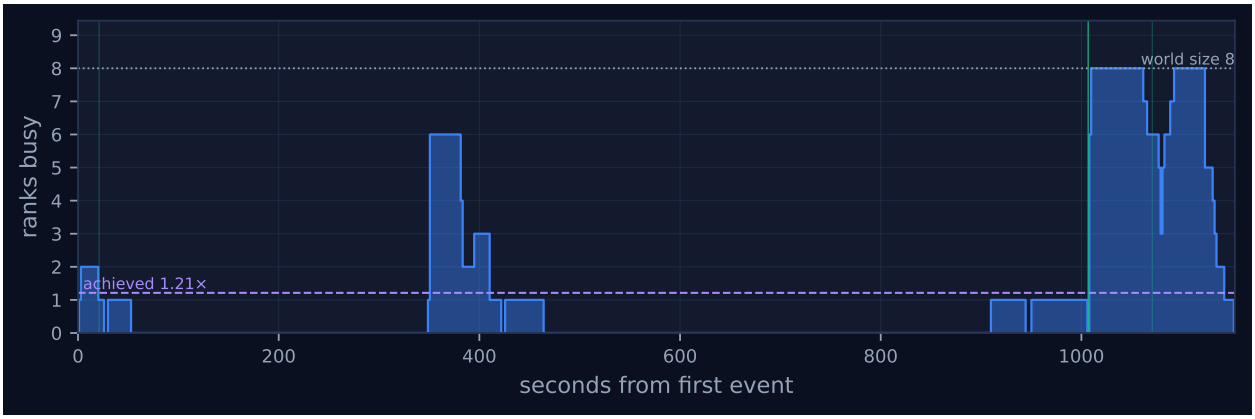


**Figure 2:** Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

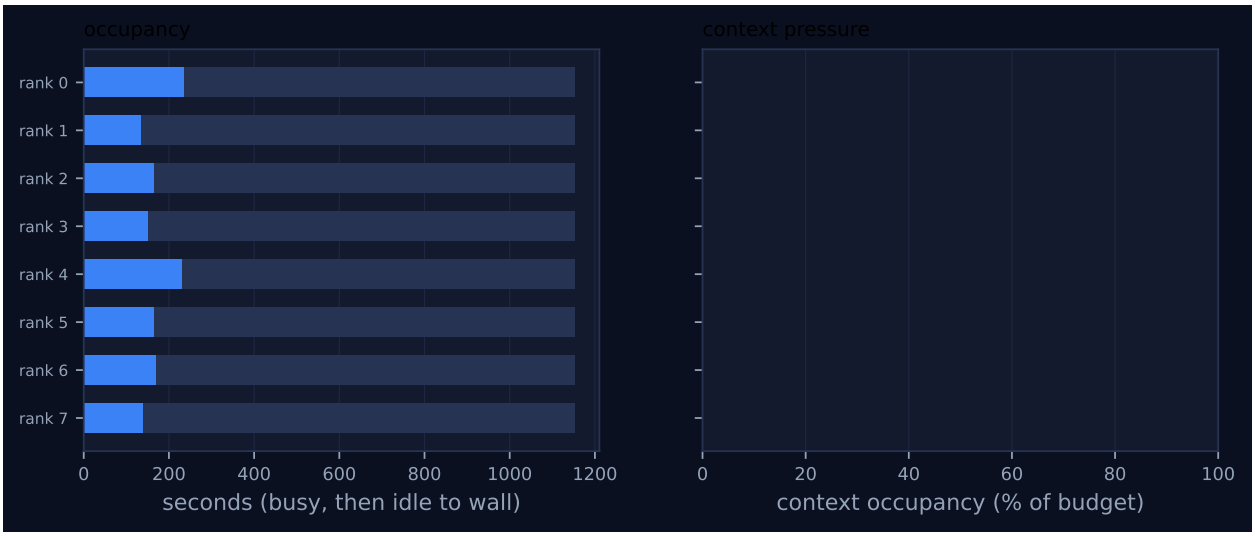


**Figure 3:** Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

# 4 Per-rank detail

| rank | busy (s) | occ. (%) | tasks | calls | sent | recv | tok. sent | ctx (%) | state |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 235.57 | 20.4 | 6 | 6 | 10 | 10 | 11,998 | 0.0 | finalized |
| 1 | 136.25 | 11.8 | 3 | 3 | 5 | 5 | 794 | 0.0 | finalized |
| 2 | 166.53 | 14.4 | 4 | 4 | 8 | 8 | 3,888 | 0.0 | finalized |
| 3 | 151.68 | 13.2 | 3 | 3 | 5 | 5 | 641 | 0.0 | finalized |
| 4 | 232.17 | 20.1 | 5 | 5 | 11 | 11 | 9,022 | 0.0 | finalized |
| 5 | 166.18 | 14.4 | 3 | 3 | 5 | 5 | 795 | 0.0 | finalized |
| 6 | 171.12 | 14.8 | 4 | 4 | 8 | 8 | 4,104 | 0.0 | finalized |
| 7 | 138.91 | 12.0 | 3 | 3 | 4 | 4 | 664 | 0.0 | finalized |

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

# 5 Collectives, against the cost model

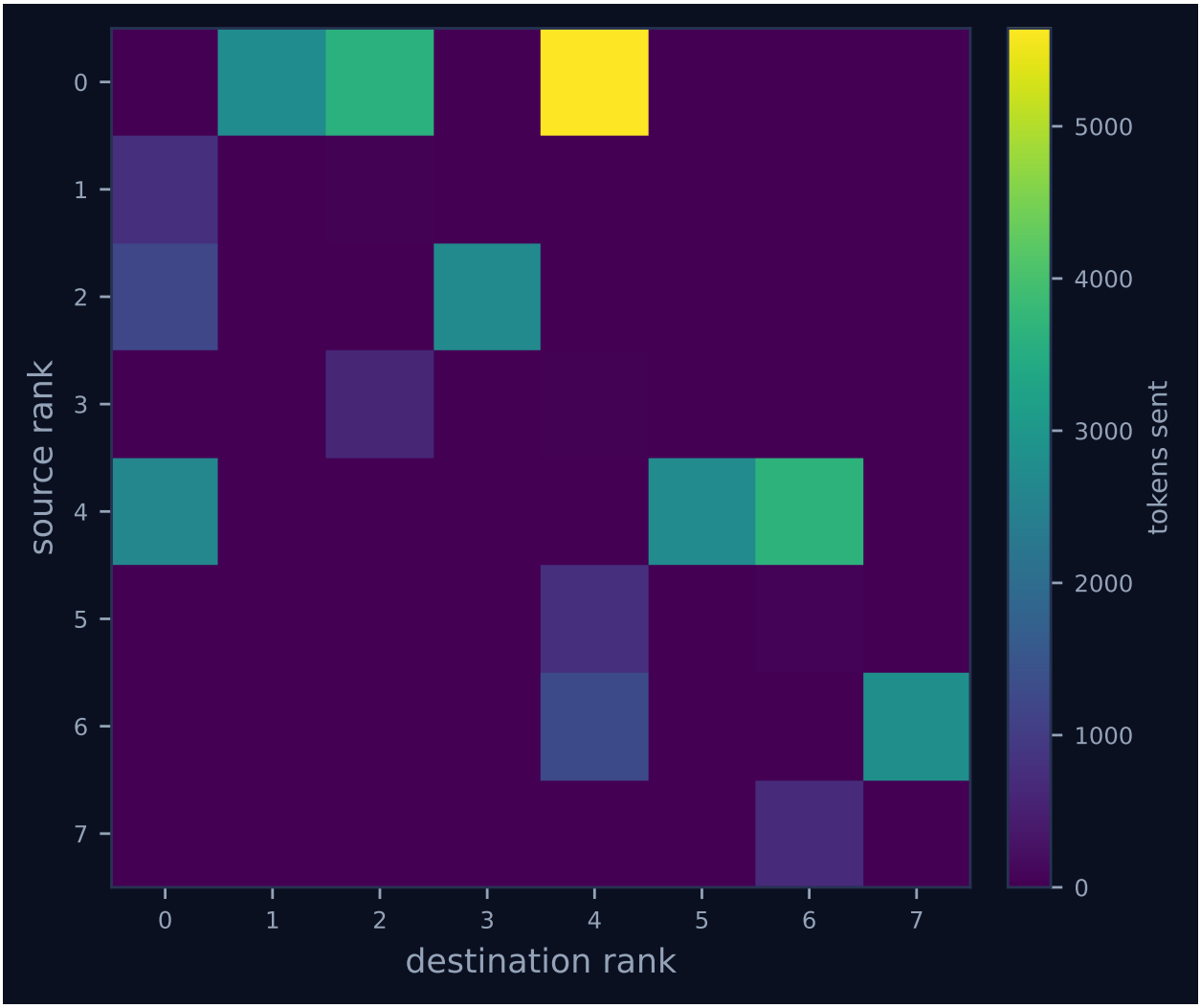


**Figure 4:** Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

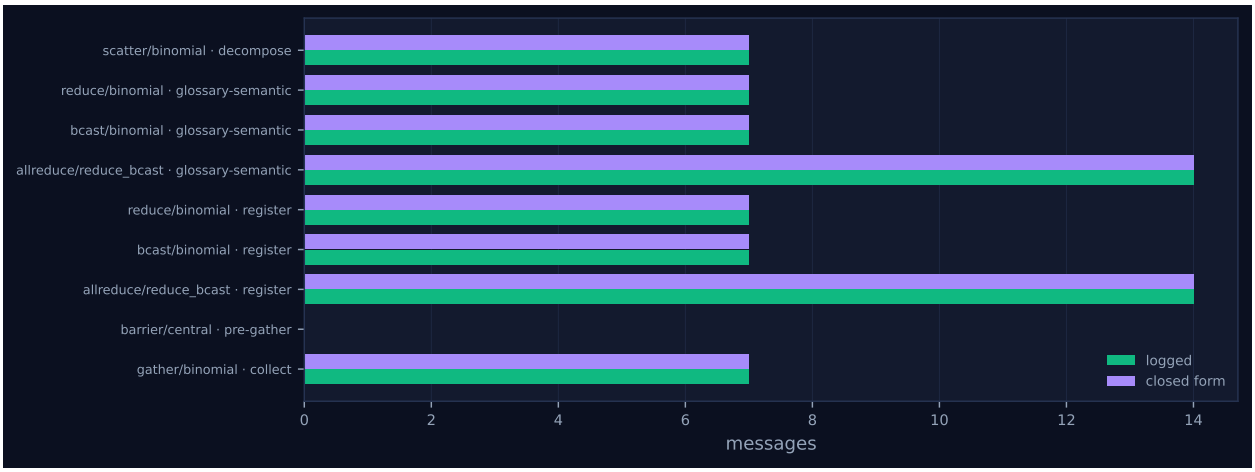


**Figure 5:** Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

| op | algorithm | label | $p$ | seen | rounds | pred. | msgs | pred. | tokens | wall (s) | skew (s) | mo |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| scatter | binomial | decompose | 8 | 8 | 3 | 3 | 7 | 7 | 11,594 | 0.02 | 0.01 | ✓ |
| reduce | binomial | glossary-semantic | 8 | 8 | 3 | 3 | 7 | 7 | 2,332 | 980.70 | 985.68 | ✓ |
| bcast | binomial | glossary-semantic | 8 | 8 | 3 | 3 | 7 | 7 | 10,780 | 985.70 | 0.10 | ✓ |
| allreduce | reduce_bcast | glossary-semantic | 8 | 8 | 6 | 6 | 14 | 14 | 0 | 985.70 | 0.10 | ✓ |
| reduce | binomial | register | 8 | 8 | 3 | 3 | 7 | 7 | 281 | 0.13 | 0.11 | ✓ |
| bcast | binomial | register | 8 | 8 | 3 | 3 | 7 | 7 | 1,295 | 0.13 | 0.03 | ✓ |
| allreduce | reduce_bcast | register | 8 | 8 | 6 | 6 | 14 | 14 | 0 | 0.13 | 0.03 | ✓ |
| halo_exchange | ? | seams | 8 | 8 | 0 | – | 14 | – | 0 | 0.00 | 18.00 | – |
| barrier | central | pre-gather | 8 | 8 | 0 | 2 | 0 | 0 | 0 | 30.20 | 0.23 | ✓ |
| gather | binomial | collect | 8 | 8 | 3 | 3 | 7 | 7 | 5,059 | 0.06 | 0.27 | ✓ |

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

## 6 Reading the timeline

Figure 1 does not look like a parallel run. Eight lanes are drawn and the blue bars in them occupy three narrow islands separated by two long stretches in which nothing at all is happening. Figure 2 makes the same point as a step function: one or two ranks busy from 1.7 to 52.4 s, six ranks busy for roughly forty seconds around 350–390 s decaying to one by 464 s, a single rank from 910 to 1,006 s, and only then the full eight from 1,007 to 1,152 s. Taking the union of every claim-to-complete interval in the log gives 395.6 s of wall clock with at least one rank executing, so 757 s of the 1,152.9 s span had no work in flight anywhere in the world. Two windows account for 742.3 s of that: 52.4 s – 348.5 s (296.2 s) and 464.2 s – 910.3 s (446.1 s).

Those two windows are the run's dominant feature and they are not the semantic operator. The broker log separates queueing from service exactly: every agent call has a `broker.enqueue`, a `broker.claim`, and a `broker.complete`. Summing over all 31 calls gives 2,653.6 s of queue wait against 1,398.4 s of service, so 65.5% of total agent-call elapsed time was spent waiting for a worker

to pick the task up. In `real-tr-p8-full` the same sum is 39.2 s of queue against 1,033.2 s of service, 3.7%. Two stalls dominate. Six of the eight `terms:u*` calls were enqueued within 30 ms of the scatter and not claimed until $t \approx 348$–351 s, because ranks 2 through 7 had no worker attached before then — the first `rank.init` with `executor: worker` for those ranks is at 348.5–351.0 s, whereas in `real-tr-p8-full` all eight workers attached within 1.6 s. The second stall is `reduce:GLOSSARY_MERGE:d2` on rank 0, enqueued at 411.0 s and not claimed until 910.3 s: 499.3 s of pure queueing on the critical path, in which rank 0 held the partially folded glossary and every other rank was blocked in the collective behind it. Rank 0's incarnation counter tells the story — incarnation 4 attached at 56.4 s and never claimed anything, and incarnation 5 did not appear until 910.3 s.

This has a direct consequence for how the viewer's statistics row should be read, and the consequence is not confined to this run. The row reports agent latency p50/p95 of 59.5/388.1 s and `metrics.json` gives a maximum of 534.17 s, which is the `reduce:GLOSSARY_MERGE:d2` call. Only 34.8 s of that 534.17 was model service; the remaining 499.3 s was queue. The `agent.call` event's `latency_s` field is measured from the harness's point of view — submission to result — so wherever workers are scarce it is a scheduling metric wearing a model-latency label.

The cost model inherits that contamination through its calibration path, and this is a general caution rather than a local one. `ampi.cost.calibrate` fits latency $= \alpha + n_{\text{out}}\beta$ by least squares over exactly these `agent.call` `latency_s` values, so every run whose broker queue is loaded calibrates a cost model on queueing rather than on generation. On this run the effect is severe enough to be self-diagnosing: the regression returns a *negative* slope, $\beta = -0.108$ s/token, because the longest queue waits happened to land on the small-output term-sheet calls while the largest-output call in the run — the 1,540-token depth-3 merge — was claimed promptly. `calibrate` has a guard for exactly this, and it fired: the $\alpha = 59.521$ s and $\beta = 0.076114$ s/token in `metrics.json` are not a fit at all but the median-based fallback, $\alpha = \text{median(latency)}$ and $\beta = \text{median(latency)}/2\,\text{median}(n_{\text{out}}) = 59.521/(2 \times 391)$. Refitting the same regression on service intervals instead (`broker.claim` to `broker.complete`) gives a well-behaved $\alpha = 39.9$ s and $\beta = 0.0117$ s/token, and moves the $\alpha/\beta$ crossover from the reported 782 tokens to 3,406. The guard prevented a nonsensical model from being published, but it did not prevent a wrong one: neither the reported $\beta$ nor the crossover should be believed for this run, and the same failure mode will silently affect any calibration taken from a run with broker contention.

The second island, 348–464 s, is where the reduction tree is visible. Six ranks finish their term sheets almost simultaneously, and then the binomial fold appears as a staircase: rank 2 and rank 6 each perform a depth-1 merge (410.9 s and 411.2 s), rank 4 performs its depth-2 merge (464.7 s), and the tree narrows to rank 0. Rank 4's lane in Figure 1 is visibly the widest bar of that island because it did three consecutive agent calls — its own term sheet, its depth-1 fold, and its depth-2 fold. The fan-in shape is also legible in Figure 4: traffic is confined to the binomial edges $0\to1$, $0\to2$, $0\to4$, $2\to3$, $4\to5$, $4\to6$, $6\to7$ and their reverses, with the brightest cell $0\to4$ at 5,644 tokens, most of it the 1,540-token merged glossary travelling down the broadcast tree.

The third island is the run behaving as designed. From 1,006.9 s all eight ranks translate concurrently, then exchange seam boundaries and revise; Figure 2 sits at 8 for most of that stretch, and the dip in the middle is the gap between the last translation completing and the last halo message arriving. The whole productive part of the harness — translate, halo exchange, revise, barrier, gather — takes 146 s and looks exactly like the corresponding phase of the exact run.

One asymmetry in the collective table repays attention. The reduce reports `wall_s` of 980.70 s and the broadcast 985.70 s, which is not two long operations but one: a leaf rank leaves the reduce in

2 ms, having only sent, and then blocks inside the broadcast until the root's answer arrives, so the leaves' waiting is charged to the `bcast` and the root's folding to the `reduce`. Ranks 1, 5, 7 and 3 record `reduce` walls of 1.4–2.2 ms and `bcast` walls of 985.7, 625.2, 624.2 and 618.7 s respectively. Added up over participants, the glossary reduce and broadcast account for 5,699.4 rank-seconds, 97% of the 5,861.7 the whole run charges to coordination, and the two dead windows fall entirely inside them.

# 7 Interpretation

## 7.1 What is true by construction

Three of this run's most striking numbers were fixed the moment the configuration was chosen. The seven extra agent calls relative to `real-tr-p8-full` are not an empirical finding: a binary reduction over $p = 8$ has $p - 1 = 7$ internal nodes, and `semantic_op` builds a kernel that invokes an agent once per node, so the count is forced. The log confirms the correspondence exactly — the seven calls whose `kind_label` begins `reduce:GLOSSARY_MERGE` are the only agent calls this run has that `real-tr-p8-full` does not, and the other 24 (`terms`, `translate`, `revise`, eight of each) match one for one. The maximum fold depth of 3 is $\lceil \log_2 8 \rceil$, and the labels record the tree level directly: four calls at `:d1`, two at `:d2`, one at `:d3`. And the byte-identity of the result at every rank follows from the algorithm rather than from the operator: `reduce_bcast` computes one distinguished answer at the root and ships it, so `allreduce` correctly records `divergence_risk: false` on all eight of its events even though the underlying `coll.reduce` events all carry `lossy: true`. The two flags mean different things and the trace keeps them apart properly. The per-rank statistics agree: all eight ranks report `glossary_digest c908d2c6e40f`, and the seven broadcast `msg.send` events carry that single digest.

## 7.2 What the fold actually did

The interesting result is what *emerged*, and it is recoverable in full because every accumulator that crossed a wire was stored in the blob store. The four leaf contributions travelled as `msg.send` payloads with an `acc` field, the four remaining leaf term sheets and all seven fold outputs are stored as blobs, and matching them by key count and containment reconstructs the entire tree. Table 1 is that reconstruction; the service times come from `broker.claim` to `broker.complete` and the output token counts from the corresponding `agent.call` events.

| node | depth | $\lvert L \rvert$ | $\lvert R \rvert$ | $\lvert L \cup R \rvert$ | $\lvert \text{out} \rvert$ | conflicts | unforced | service (s) | out tok. |
|---|---|---|---|---|---|---|---|---|---|
| rank 0 | 1 | 38 | 27 | 62 | 62 | 0 | 0 | 22.8 | 490 |
| rank 2 | 1 | 31 | 25 | 50 | 50 | 1 | 2 | 21.2 | 362 |
| rank 4 | 1 | 43 | 24 | 59 | 59 | 0 | 0 | 26.4 | 456 |
| rank 6 | 1 | 31 | 29 | 55 | 55 | 1 | 0 | 23.7 | 403 |
| rank 0 | 2 | 62 | 50 | 106 | 106 | 2 | 0 | 34.8 | 809 |
| rank 4 | 2 | 59 | 55 | 108 | 108 | 1 | 3 | 39.0 | 813 |
| rank 0 | 3 | 106 | 108 | 203 | 203 | 1 | 0 | 56.6 | 1,540 |

**Table 1:** The seven applications of `GLOSSARY_MERGE`, reconstructed from the blob store. "conflicts" counts keys present in both inputs with disagreeing values; "unforced" counts keys the operator rewrote although its two inputs did not disagree about them.

The column that matters is |out| against $|L \cup R|$. They are equal at every node. Across all seven folds, at depths 1, 2 and 3, the operator dropped zero keys and invented zero keys, and the 203 terms in the final glossary are exactly the 203 distinct source terms the eight ranks proposed between them. For an operator the runtime classifies as lossy (`Associativity.APPROX`, hence `Op.lossy` true, hence `lossy: true` on every `coll.reduce` event), the measured key-level loss on this workload was nil. The declaration is a worst-case contract, not a measurement, and this run is a data point saying the worst case did not occur.

Nothing about depth compounded either, and the reason is visible in the token counts. The operator was not summarising: its output grew with its input at a steady $\approx 7.5$ tokens per term (490 tokens for 62 terms, 809 for 106, 1,540 for 203). The budget formula in `semantic_op` gives $\max(300, 1500(1 + 0.25d))$, so 2,625 tokens at depth 3, and the root fold used 1,540 of them — 59% headroom. Depth compounds loss for a lossy operator only when each level forces a re-encoding under pressure; here every level had room to copy, so the mechanism the module docstring warns about never engaged. That is a scale-conditional result, not a refutation, and Section 8 says where it stops holding.

### 7.3 Does the result actually differ from the exact one?

Yes, and the difference is measurable without leaving this run. Comparing against `real-tr-p8-full`'s stored glossary is tempting — 203 terms versus 206, with 16 keys unique to the semantic run and 19 unique to the exact one — but that comparison is worthless, because the eight term sheets themselves were produced by nondeterministic agent calls and differ between the two runs. The clean experiment is a within-run counterfactual: take this run's own eight term sheets from the blob store, fold them with `agentmpi.ops.UNION`, and apply the harness's deterministic tiebreak (`sorted(v)[0]` on each conflict list), which is precisely what `real-tr-p8-full` did to its own sheets.

That replay produces 203 keys — *the identical key set* — and 7 values that differ from the semantic result, 3.4% of the glossary. Exactly five terms attracted conflicting proposals from the population (*German*, *Irene Adler*, *brougham*, *cabman*, *reasoner*). The semantic operator resolved all five by choosing one of the two renderings that had actually been proposed and never invented a third, and it chose what `UNION`'s lexicographic tiebreak would have chosen on three of the five; *German* (country versus demonym) and *cabman* ("driver" versus "carriage driver") are the two it decided differently. The remaining five differences are edits made to terms whose two inputs did not disagree at all: *Bohemian* and *European* were moved from bare place nouns to attributive forms, *mistress* from "kept woman" to "lady of the house", and — the interesting pair — *Miss Adler* and *Miss Irene Adler* were rewritten to use the transliteration of the surname the operator had already chosen for *Irene Adler*.

That pair is the whole argument for semantic reduction, in miniature and on real data. `UNION` resolved *Irene Adler* the same way the model did, but it treats *Miss Adler* as an unrelated key and left it spelling the surname the other way, because set union has no notion that two keys are about the same person. The semantic operator noticed and propagated. Seven agent calls and $0.0894 therefore bought seven changed glossary entries: two cross-key consistency repairs an exact operator cannot express, two conflict resolutions decided against `UNION`'s arbitrary tiebreak, and three unforced substitutions whose merit is a matter of translation judgement rather than measurement.

The order-dependence the operator is warned about is also visible, in one instance. Table 1 counts

six conflicts across the tree but the population only ever disagreed about five terms; the sixth is *Bohemian* at rank 0's depth-2 fold, and it exists only because rank 2's depth-1 fold had unilaterally rewritten that term one level earlier. A lossy operator does not merely resolve the disagreements it is given, it manufactures new ones for the levels above it, and under a different tree shape that particular conflict would never have arisen.

The harness's own quality scoring cannot see any of it. Coverage, consistency, divergent entities, rendering verification, paragraph fidelity and length plausibility are identical between this run and `real-tr-p8-full` — 1.0, 1.0, 0, 1.0, 1.0, 1.0 — and identical again in `real-tr-p8-noglossary`, which skips the allreduce entirely. With six canonical shared entities across eight sections the metric is saturated, so no conclusion about translation quality can be drawn in either direction from this ablation as scored.

### 7.4 The cost, attributed

Against `real-tr-p8-full`: agent calls 31 versus 24, prompt tokens 48,530 versus 43,448, completion tokens 13,878 versus 8,636, cost \$0.3538 versus \$0.2599, messages 56 in both. Priced at the run's own calibration (\$3/Mtok in, \$15/Mtok out), the seven merge calls alone account for \$0.0894 of the \$0.0939 difference, so cost attribution to the operator is essentially exact. Latency is where the naive comparison misleads. The reduce's recorded `wall_s` went from 3.23 s to 980.70 s, a factor of 303, and the enclosing allreduce from 10.24 s to 985.70 s, a factor of 96. But rank 0's 980.7 s inside the reduce decomposes into 114.2 s of its own model service, 507.7 s of broker queue (3.6 + 499.3 + 4.8 across its three folds), and 358.6 s spent waiting for rank 2's depth-1 contribution, which was itself stuck behind rank 2's 348.5 s wait for a worker. So 866.3 s — 88% — traces to worker attachment, directly or through a peer. The operator's own contribution is 224.5 s of model service across the seven calls, of which the critical path through the tree, $\max(\max(26.4, 23.7) + 39.0,\ \max(22.8, 21.2) + 34.8) + 56.6$, is 122.0 s. Service time did grow with depth, roughly in proportion to output size: 23.5 s mean at depth 1, 36.9 s at depth 2, 56.6 s at depth 3. A well-scheduled version of this run would therefore show a glossary phase of about two minutes rather than the sixteen recorded, against a genuinely free fold in the exact run.

### 7.5 The flagged findings

The [warning] on `halo_exchange` is a real defect in AgentMPI, and it is not specific to this configuration. The collective reports zero messages while the fabric logged 14 carrying its internal tag. Inspecting the recorded event shows the payload is `{label, left, right}` only: no `algorithm`, `size`, `rounds`, `messages_sent`, `tokens_sent` or `wall_s`. The same warning appears in `real-tr-p8-full`, `real-tr-p8-rdallreduce`, `real-tr-p4-full` and `real-tr-p2-full`, and is absent only from `real-tr-p8-nohalo`, which does not call the operation. It has since been diagnosed and fixed — commit `ee21d79` adds those fields to all three neighbourhood collectives and asserts on the trace rather than on the runtime counter — but the trace analysed here was recorded at commit `a6f6103`, before the fix, so the flag is accurate for this artefact and the derived `halo_exchange` row in the collective table (0 messages, 0 tokens, 0 s, algorithm `?`) should be read as missing data rather than as measurement.

The [note] on 15.2% parallel efficiency and 84.8% idle rank-seconds is real but misattributed if read as a property of semantic reduction. The reduction is genuinely serial — a $\log_2 p$ tree keeps at most four ranks busy at level 1 and one at level 3 — but the efficiency collapse is dominated by the

742.3 s in which no rank at all was busy. The exact run with the identical harness scores 77.4%.

The [note] on a 63.6% coordination share is real, and it needs unpacking, because on its face it says the opposite of this section's conclusion. Read naively — two-thirds of the population's rank-seconds inside collectives — it invites the reading that coordination is the run's cost and that the semantic operator, being the thing inside the collective, is the cause. The decomposition says otherwise, and separating those two is the sharpest thing this run contributes.

Of the 5,861.7 rank-seconds charged to coordination out of the $8 \times 1{,}152.9 = 9{,}223.2$ available, 5,699.4 are inside the glossary reduce and broadcast. Intersecting each rank's blocking interval with the broker records for the seven merge tasks splits that figure three ways: 224.5 rank-seconds (3.9%) with a rank actually executing a merge, 521.8 (9.2%) with a rank holding a merge task that no worker had yet claimed, and 4,953.1 (86.9%) with a rank blocked waiting for a peer. Across all primitive collectives, 4,245.5 of the 5,861.7 rank-seconds — 72.4% — elapsed during moments when *no rank anywhere in the world was executing anything*, and 4,160.9 of those fall inside the two long dead windows. So the coordination share is high and the semantic fold is only 3.8% of it.

That both strengthens and sharpens the wall-time argument. It strengthens it because the same conclusion now follows from a bounded, additive measure rather than from a wall-clock subtraction: coordination dominates the rank-seconds, but coordination here overwhelmingly means *blocked on an absent worker*, not *running a language model*. It sharpens it because the companion `coordination_span_share` of 88.1% locates the two dead windows precisely. Coordination was in flight somewhere for 1,016.1 s of the 1,152.9 s wall, and the 757.3 s during which nothing ran is almost entirely a subset of that: the run's idleness is not idleness *between* collectives, it is idleness *inside* the glossary allreduce, with up to eight ranks parked in a collective that could not advance because its root had a queued task and no process to run it.

The comparison against the exact run confirms the attribution, since both figures are now proportions on the same scale. Recomputing the same quantity from `real-tr-p8-full`'s trace with today's `agentmpi.analysis` gives a coordination share of 16.4% on 219.2 rank-seconds, of which its glossary reduce and broadcast are only 52.2 — against 5,699.4 here, a factor of 109 — while its `pre-gather` barrier accounts for 163.1, three quarters of its total. `real-tr-p8-rdallreduce` sits at 16.1% and `real-tr-p8-noglossary`, which has no glossary collective at all, at 10.1%. Substituting a semantic operator therefore did move the coordination share by a factor of four, and the honest statement of why is that it converted a free fold into seven scheduled tasks on a scheduler that could not keep up, of which the model work itself was two hundred rank-seconds.

The two [ok] findings are both correct and both worth keeping. Fifty-six reattachments, five to eight per rank and reaching incarnation 9 on ranks 0, 1, 4 and 6, is nearly double the 31 of `real-tr-p8-full`, and it is the same phenomenon as the queue stalls seen from the other side: each rank role outlived a succession of worker processes, and its mailbox, sequence numbers and context account survived every handover. Nothing was lost across an attachment boundary — 56 sends, 56 receives, zero failures, zero contract violations, zero retries. And all nine checkable collectives sent exactly the message count their closed forms predict.

### 7.6 Position in the ablation ladder

The harness docstring predicts that a semantic merge "would be none of those [exact, associative, commutative, idempotent], and the population would end up believing $p$ slightly different glossaries". The first clause remains a correct statement about the operator's declared algebra; the second did not happen and, under `reduce_bcast`, cannot. The population divergence the docstring warns

about is a property of the *algorithm*, not the operator: `recursive_doubling` has every rank perform its own fold sequence, which is why `algorithms.allreduce` passes `divergence_risk=op.lossy` on that path and a hardcoded `False` on the `reduce_bcast` path.

The sibling `real-tr-p8-rdallreduce` does not test that combination. Its configuration is `glossary_op: union` with `allreduce_alg: recursive_doubling`, so its operator is exact and its 16 `coll.allreduce` events all record `divergence_risk: false` — as they should, since recursive doubling with a canonical operand ordering over an exact operator gives every rank the same answer. No run in `traces/events/` sets `divergence_risk` true; the semantic-operator-plus-recursive-doubling cell of the design is empty. This run therefore supplies the cost of a semantic operator and the demonstration that `reduce_bcast` contains it, and says nothing at all about divergence. That cell is where the project's divergence claim would have to be measured, and the two runs that exist do not measure it between them.

## 8 Threats to this reading

**The wall-time comparison against `real-tr-p8-full` is not controlled.** Both runs executed the same harness against real Cursor subagents, but this one lost 742.3 s to worker attachment stalls and the other lost 2.3 s. Nothing in the configuration explains the difference; the semantic operator does not spawn workers. Any sentence of the form "the semantic glossary made the run 6.9× slower" is contaminated by an environmental variable neither run controlled, and the honest operator-attributable latency figure is the 122.0 s critical path computed from service intervals. That figure in turn assumes queueing and service are cleanly separable in the broker log, which they are only if a claimed task begins executing immediately.

**The zero-loss result is conditional on scale and on budget headroom, and this run cannot tell which.** The root fold produced 1,540 tokens against a 2,625-token budget. The glossary size is set by the book, not by $p$ — the $p = 1$, $p = 2$, $p = 4$ and $p = 8$ union runs report 208, 207, 208 and 206 terms — so at larger $p$ the same $\approx 200$-term payload would pass through $\lceil \log_2 p \rceil$ folds with a budget that grows only as $1500(1 + 0.25\lceil \log_2 p \rceil)$. At $p = 8$ that is comfortable. It is also possible that this particular model would have copied faithfully at any budget and the headroom is irrelevant. One run at one $p$ cannot separate those.

**The seven-value difference is one sample of a nondeterministic operator.** The whole point of `semantic_op` is that it is not reproducible, and this run folded the tree once. A second execution over the same eight term sheets would give a different set of edits, possibly a different count, and possibly a nonzero key loss. The 3.4% figure is a single draw, not an expectation, and the run contains no repeat from which to estimate variance.

**"Better" is asserted, not measured.** I judged two of the seven edits to be genuine cross-key consistency repairs and the rest to be plausible translation choices, by reading them. The harness's quality scorer is saturated — identical values on every metric for the semantic, exact and no-glossary runs — so there is no instrument in this experiment that can distinguish a semantic merge from no merge at all on output quality. Any claim that the $0.0894 bought something is currently a claim about the glossary, not about the book.

**The counterfactual replay reuses code that has moved on.** The `UNION` fold I ran to construct the comparison glossary is today's `src/agentmpi/ops.py`, whereas the trace was recorded at commit `a6f6103`. `UNION` is a small and stable function and the `real-tr-p8-full` glossary it produced at run time is consistent with what today's version produces, but the replay is a reconstruction rather

than a recording, and the seven-value difference inherits that caveat.

**The project's divergence claim is unmeasured, and this run does not measure it.** The danger a lossy reduction operator is supposed to pose is that the population ends up holding different answers. That failure requires `recursive_doubling`, where each rank performs its own fold sequence; `algorithms.allreduce` passes `divergence_risk=op.lossy` only on that path. This run pairs a semantic operator with `reduce_bcast` and therefore cannot diverge, and `real-tr-p8-rdallreduce` pairs `recursive_doubling` with the exact `UNION` and therefore also cannot. Grepping every log in `traces/events/` finds no event anywhere in the archive with `divergence_risk` true. The semantic-operator-with-recursive-doubling cell is empty, so the claim the semantic-reduction argument rests on has never been exercised, let alone quantified. Nothing in this document should be read as evidence for or against it.

**Some derived metrics still do not mean what they appear to.** Agent latency p95 and maximum are dominated by broker queueing, and $\alpha$ and $\beta$ are derived from those latencies — on this run via the median fallback rather than a regression, because the regression's slope came out negative. The `halo_exchange` row is empty because of the instrumentation gap rather than because no traffic occurred. `tokens_deferred` and `tokens_unadmitted` now report 8,893 and 31,906 respectively, and their sum is exactly the 40,799 the `job.finish` event recorded, which is the signature of the double count that has since been separated: every message in this harness is received with `admit=False`, so the eight rendezvous transfers were charged once on send and every transfer once again on receive. No token or cost figure in this document was taken from `job.finish` — the $0.0894 attribution and the 5,441/4,873 merge token counts come from `agent.call` events and the totals from `cost.summarise`, which was never affected — but the `job.finish` value survives in `results/translation/real-tr-p8-gloss-halo-semantic-reduce_bcast-w600-u8.json`, so a reader taking transport figures from the results file rather than from `metrics.json` will still get the inflated number. None of this touches the operator findings, which rest on event counts, blob contents and agent-call token counts rather than on derived timings or transport accounting.